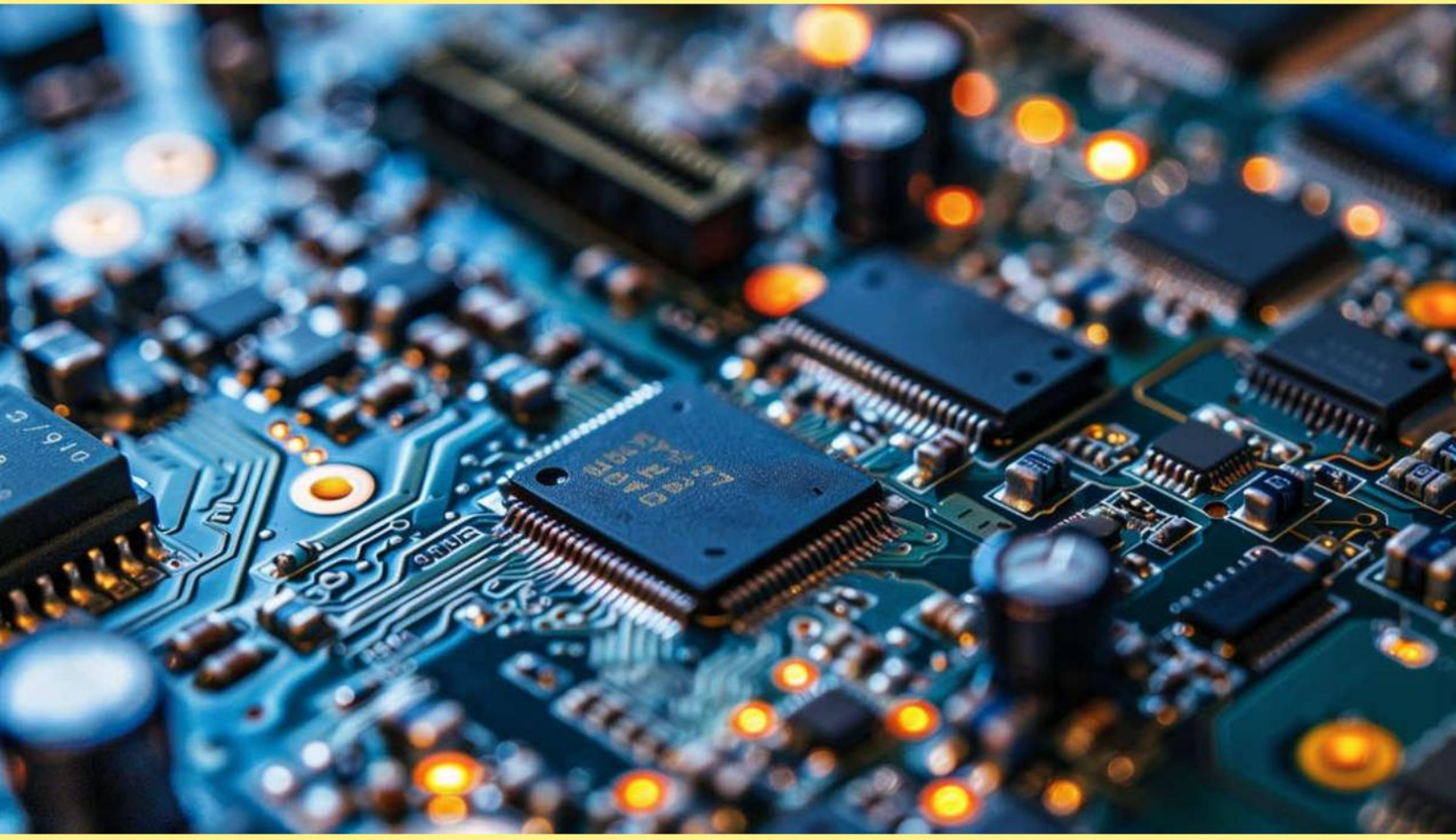
CPI A2 – PROJET SYSTÈMES EMBARQUÉS

PROSIT 5 – PILE



CESI
ÉCOLE D'INGÉNIEURS

Contexte :

Le projet consiste à optimiser un programme en C sur la carte NUCLEO-L476RG en étudiant l'utilisation de la mémoire et la taille de la pile d'exécution.

Mots inconnus :

Pile (Stack) : zone de la mémoire RAM où sont empilés, à chaque appel de fonction, l'adresse de retour, les paramètres et les variables locales, puis libérés au retour de la fonction.
Tas (Heap) : zone de la mémoire RAM réservée aux allocations dynamiques, faites pendant l'exécution du programme avec malloc() et libérées avec free().
Variables locales : variables déclarées à l'intérieur d'une fonction, stockées dans la pile et qui n'existent que pendant l'exécution de cette fonction.
static : mot-clé du C qui fait garder sa valeur à une variable locale entre deux appels, en la plaçant hors de la pile (zone des données globales).
Linker script : fichier qui indique à l'éditeur de liens où placer chaque partie du programme (code, données, pile, tas) dans la mémoire du microcontrôleur.
malloc() : fonction du C qui réserve dynamiquement un bloc de mémoire dans le tas et renvoie un pointeur vers ce bloc.

Problématique :

Comment réduire la taille de la pile d'exécution ?

Plan d'action :

- Étudier l'organisation de la mémoire [20 min]
- Identifier les rôles de la pile lors de l'appel à une fonction [30 min]
- Identifier les différentes zones mémoire : heap / stack, point attaque, point BSS et savoir ce que contient chacune [1h]
- Simuler le nouvel algo transformé en C [1h30]
- Calculer la taille de la pile utilisée [20 min]
- Comparer l'avant-après [15 min]

Réalisation :

Cadrage du travail

Lors de l'analyse du sujet, nous avons noté les éléments suivants :

Pb : comment réduire la taille de la pile.

Contraintes :

- sram limitée a 2Ko partagée entre les variables globales, la pile et le tas
- fixer une limite de taille pour la pile.
- temps d'acces aux variables (utilisation des pointeurs)
- rester formel au comprtement de l'algorithme.

Livrables :

- Programme.
- schéma de la pile a en cours d'évolution
- maquette physique du système embarqué

Pointeurs et passage par adresse

Pour manipuler les pointeurs, nous avons d'abord écrit un premier programme qui affiche une variable, puis la modifie au travers de son adresse :

```
1   int main(){
2       int n;
3       int *p;
4       n =5;
5       printf("%d\n",n);
6       p =&n;
7       printf("%d\n",*p);
8       *p=1;
9       printf("%d\n",n);
10      printf("%d\n",*p);
11      printf("%x %x\n",p,&n);
12      printf("%x %x\n",p+1,&n+2 );
13      return 0;
14  }
```

La fonction suivante échange deux variables en recevant leurs adresses plutôt que des copies de leurs valeurs :

```
1   void echanger(int *adr_a, int *adr_b)
2   {
3   int tmp;
4   tmp = *adr_a;
5   *adr_a = *adr_b;
6   *adr_b = tmp;
7   }
8
9   int main()
10  {
11  int a=3,b=258;
12  echanger(&a, &b);
13  printf("%d\t%d\n",a,b);
14  return 0;
15  }
```

Nouvel algorithme transformé en C

Le nouvel algorithme de mesure a été retranscrit en C. Les tableaux sont désormais des variables locales de `loop()`, passés aux procédures par pointeur, et l'indice de la moyenne glissante est une variable `static` :

```
// Constantes
#define Nb_Capteurs 5
#define Nb_Val 10

uint8_t Lecture_capteur(float *Mesure, uint8_t i);
void Lecture(float *Tab_Val, uint16_t *Erreurs);
void Add_Val(float *Tab_Moy, float Val);
void Afficher(float *Tab_Moy, uint16_t *Erreurs);

void setup()
{
  Serial.begin(9600);
  RandomSeed(analogRead(A0));
}

void loop()
{
  // Variables locales (loop)
  uint16_t Nb_erreur[Nb_Capteurs] = {0};
  float Moy_gliss[Nb_Val] = {0};

  Lecture(Moy_gliss, Nb_erreur);
  Afficher(Moy_gliss, Nb_erreur);
  delay(1000);
}

// PROCEDURE LECTURE(POINTEUR_TABLEAU , POINTEUR_ERREURS)
void Lecture(float *Tab_Val, uint16_t *Erreurs)
{

    float Mesure;
    uint8_t Erreur, i;
    for (i = 0; i < Nb_Capteurs; i++)
    {
        Mesure = 0;
        Erreur = Lecture_capteur(&Mesure,i);
        if (Erreur)
        {
            (Erreurs[i])++;
        }
        else {
            Add_Val(Tab_Val,Mesure);
        }
    }
}
// PROCEDURE ADD_VAL(POINTEUR_TABLEAU , VALEUR)
void Add_Val(float *Tab_Moy, float Val)
{
    static uint8_t ind_moy = 0;
    Tab_Moy[ind_moy] = Val;
    if (ind_moy >= Nb_Val - 1) ind_moy = 0;
    else ind_moy++;
}

// Simulation de la lecture d'un capteur par une valeur aléatoire
uint8_t Lecture_capteur(float *Mesure, uint8_t i)
{
    *Mesure = random(1000)/10.0 + i;
    return random(10) == 0;
}

void Afficher(float *Tab_Moy, uint16_t *Erreurs)
{
    uint8_t i;
```

```
65        for (i = 0; i < Nb_Val; i++)
66        {
67            Serial.print(Tab_Moy[i], 1);
68            Serial.print(" ");
69        }
70        Serial.println("| err : ");
71        for (i = 0; i < Nb_Capteurs; i++)
72        {
73            Serial.print(Erreurs[i]);
74            Serial.print(" ");
75        }
76        Serial.println();
77    }
```